Array - ArrayList

Môn học: Kỹ thuật lập trình

GV: TS. Võ Thị Hồng Thắm

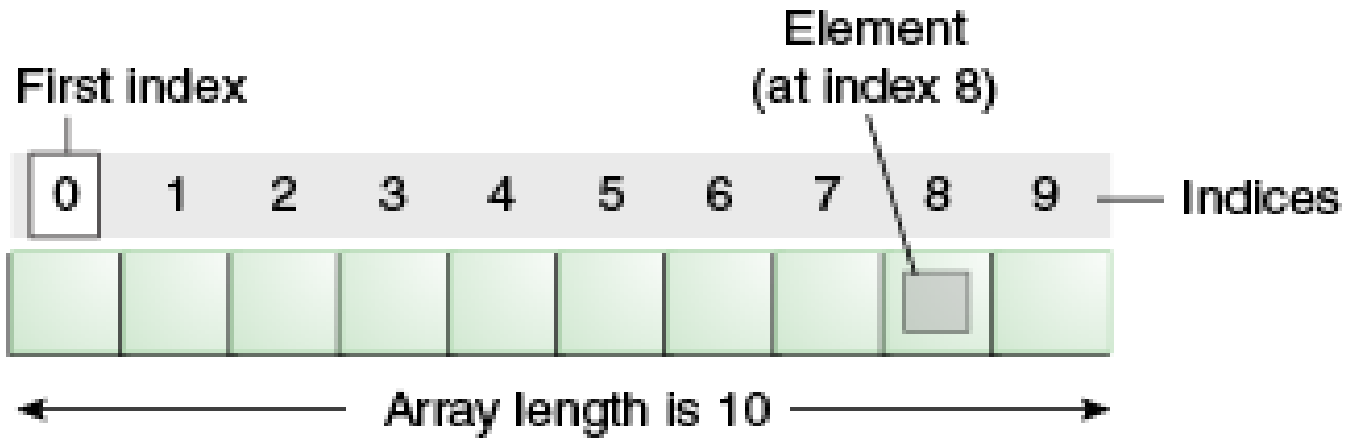
Mục tiêu

- Sử dụng array/array list để lập trình giải các bài toán

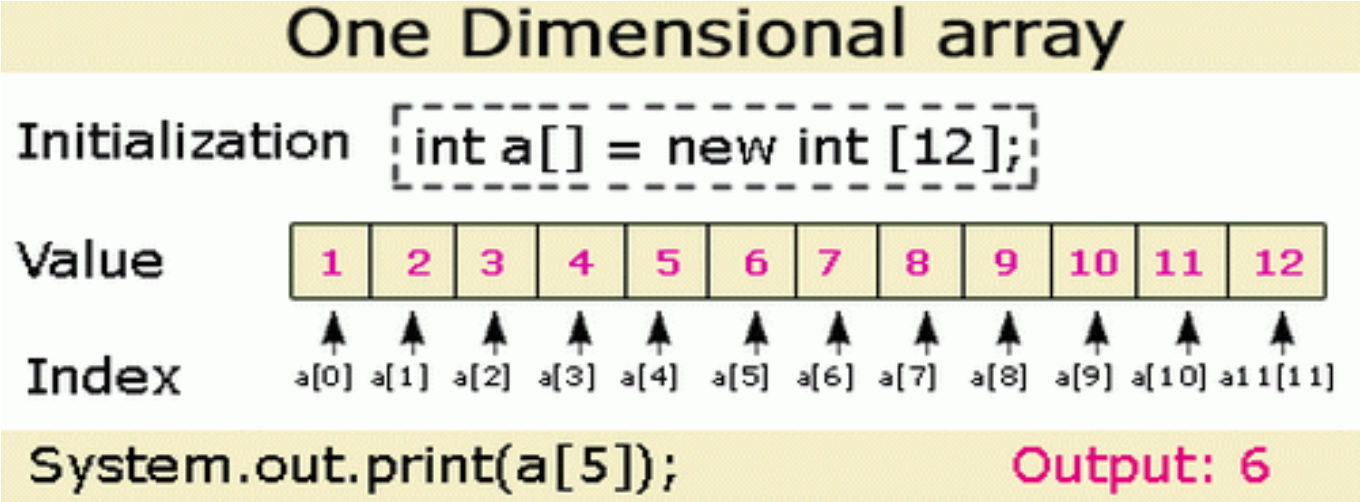
Array

Giới thiệu

- Mảng là một cấu trúc dữ liệu đơn giản trong ngôn ngữ lập trình, dùng để lưu trữ những đối tượng dữ liệu cùng kiểu, cùng loại và cùng mục đích. Khi được khai báo, mảng được cấp phát một cách tuần tự, liên tục trong bộ nhớ
- Đặc điểm đầu tiên cần ghi nhớ là: mảng lưu trữ một tập các giá trị cùng kiểu
  - ✓ Cùng kiểu "*nguyên thủy*" (int, double, etc.) or
  - ✓ Cùng loại đối tượng (***SinhVien***, ***HangHoa***, ***NhanVien***, ***PhuongTienGiaoThong***, ...)



- Trong ngôn ngữ lập trình Java, mảng là một phần quan trọng trong lưu trữ dữ liệu (*trên bộ nhớ*) & quá trình xử lý thuật toán. Một số điểm cần lưu ý khi làm việc với mảng trong Java
  - Mảng là **objects**, không phải "*dữ liệu nguyên thủy - primitives data*" giống như **int** hay **double**. Do đó, khi cần cấp phát 1 mảng trong chương trình, bạn có thể sử dụng cú pháp để tạo object. VD
    int[] a= new **int**[12]; *//--- Cấp phát 1 mảng a có 12 phần tử*
  - Ở ví dụ trên, a là một object chứa các phần tử là các số nguyên, kiểu int. Object a có "*chiều dài – length*" , do đó, ta có thể xác định chiều dài của mảng thông qua thuộc tính length như sau:
    int chieuDaiMangA = a.***length***; *//---- chieuDaiMangA = 12*



- Mỗi phần tử của mảng có thể truy xuất dựa trên chỉ số của chúng. VD, để gán giá trị cho phần tử đầu tiên & phần tử thứ 2 của mảng a, ta có thể thực hiện
  a[0]= 4; a[1]= 77;
- Chỉ số của phần tử đầu tiên của mảng luôn bắt đầu là 0, (*Không phải 1*). Như vậy, chỉ số của các phần tử thuộc mảng được mô tả như sau
  - Với 1 mảng có N phần tử thì "*chỉ số - index*" của chúng được xác định từ 0 đến N-1
  - Theo cách khác, ta nói: Mảng a có các phần tử a[0], a[1], … a[11] (*Ở VD trên, a có 12 phần tử*)
- Chiều dài, hay kích thước của 1 mảng không thể bị thay đổi sau khi đã khai báo. VD: mảng a đã khai báo có 12 phần tử, thì không thể thay đổi kích thước, cho đến khi bị loại khỏi bộ nhớ.
- Có thể khai báo đồng thời khởi gán giá trị cho các phần tử của mảng theo cú pháp sau
  int[] intArray= {5, 77, 4, 9, 28, 0, -9};
  *//--- Trong trường hợp này, không cần sử dụng từ khóa **new***
- Trong trường hợp cấp phát mảng mà không chỉ ra giá trị khởi tạo (*giống như ví dụ trước*) thì các phần tử của mảng mặc nhiên sẽ được gán =0
  int[] a= new **int**[12]; *//--- Cấp phát 1 mảng a có 12 phần tử, giá trị của mỗi phần tử =0*

Copying Arrays

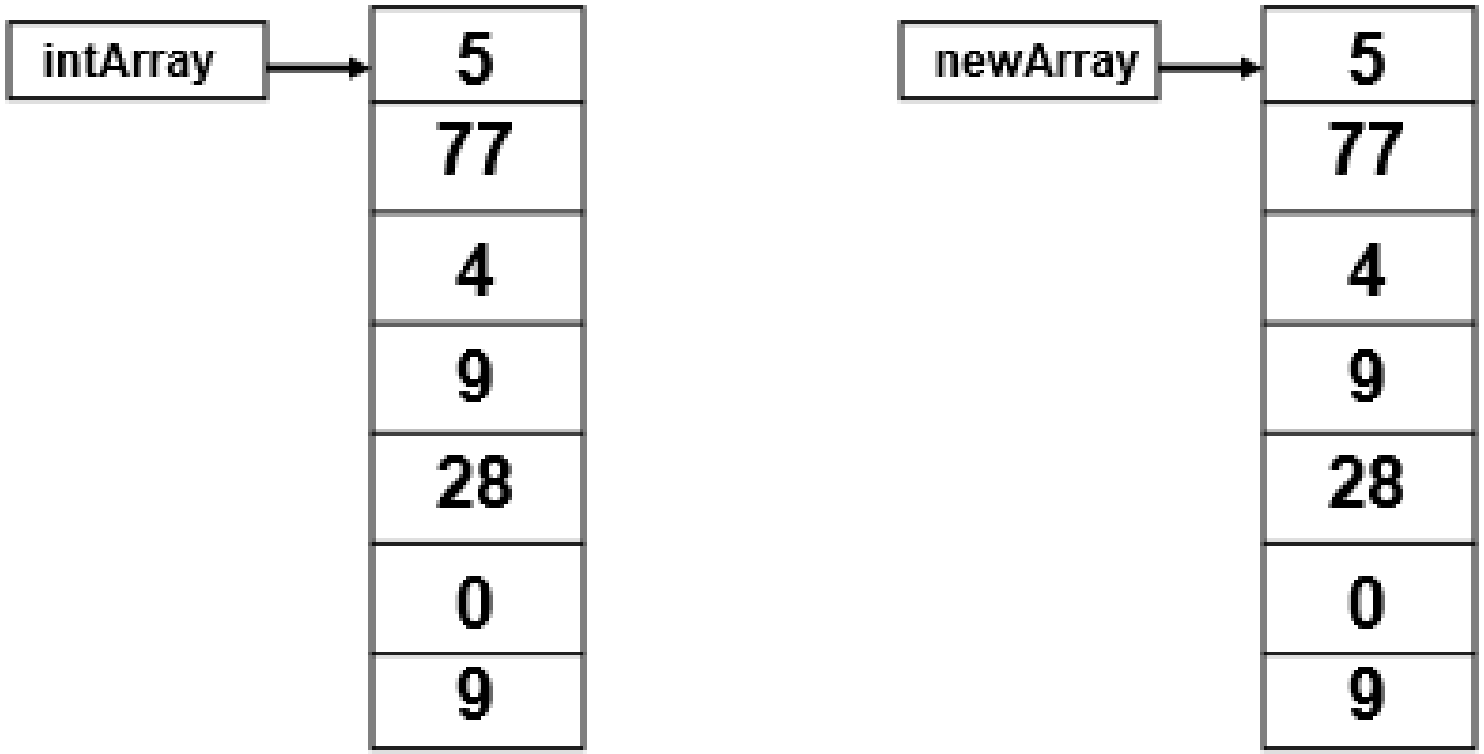
- Để sao chép 1 mảng, ta sử dụng phương thức ***arraycopy***() của System class, theo cú pháp như sau:

  public **static** void ***arraycopy***(**Object** src, **int** srcPos,
  Object dest, **int** destPos,
  int length)

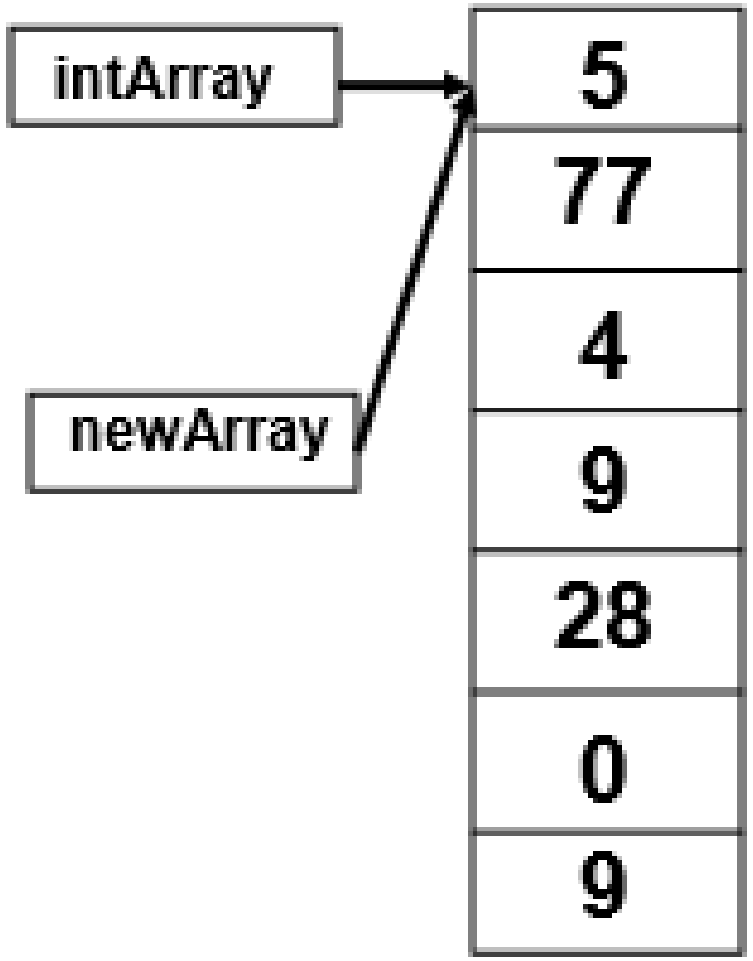
  - **src** – Mảng được xem là ***nguồn*** cho quá trình sao chép.
  - **srcPos** – Vị trí của phần tử bắt đầu sao chép (Index).
  - **dest** – Mảng được xem là ***đích*** của quá trình sao chép (*Mảng sẽ được tạo thành*).
  - **destPos** – Vị trí của phần tử bắt đầu nhận dữ liệu tại "*mảng đích*" (Index).
  - **length** – Số lượng phần tử sẽ sao chép.

Copy entire array

```
int[ ] intArray= {5, 77, 4, 9, 28, 0, 9};
int[ ] newArray = new int[intArray.length];
System.arraycopy(intArray,0,newArray,0,intArray.length)
```



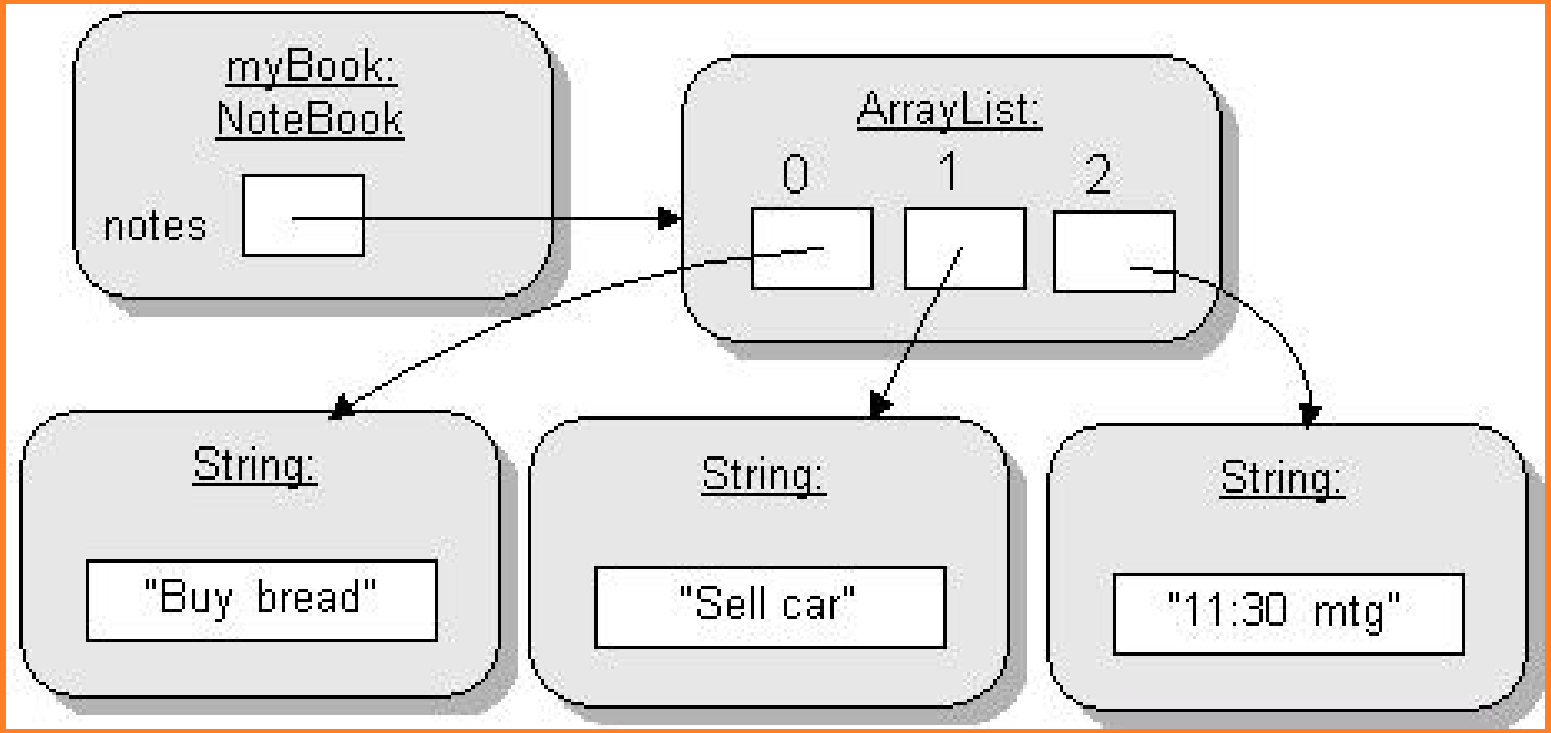
Copy reference




```
int[ ] intArray= {5, 77, 4, 9, 28, 0, 9};
int[ ] newArray = intArray;
```

ArrayList

ArrayList example in java



- **ArrayList** là một class được định nghĩa ở dạng collection, thuộc package: java.util. **ArrayList** mang nhiều đặc điểm gần giống như Array, nhưng có thêm các phương thức hỗ trợ tốt hơn so với Array trước đây
- Một điểm mới của **ArrayList** so với Array là **ArrayList** có thể thay đổi kích thước tùy thuộc vào "*nhu cầu sử dụng*" khác hoàn toàn với "*kích thước cố định*" đối với Array
  - Kích thước có thể tăng khi có nhu cầu
  - Để xác định kích thước của **ArrayList**, sử dụng phương thức size()

- ArrayList cho phép lưu trữ các phần tử khác loại
  - Mỗi phần tử có trong ArrayList được xem như là 1 Object mà nó tham chiếu đến
  - ArrayList không cho phép chứa dữ liệu "*nguyên thủy – primitive data*" như: int, double, ...)
  - Nếu cần phải chứa các giá trị nguyên, số thực, … trong ArrayList, hãy sử dụng wrapper classes

Generic

- Chỉ ra loại đối tượng cụ thể cần lưu trữ trong collection. Điều này mang lại 3 lợi ích rất lớn trong lập trình
  - **Type-safety :** Chỉ cho phép lưu giữ 1 loại đối tượng trong collection tại một thời điểm. Điều này khác hẳn với việc lưu trữ "tùm lum" loại dữ liệu trong cùng một nơi chứa
  - **Type casting is not required:** Không cần phải casting do mỗi collection khi sử dụng, chỉ lưu trữ 1 loại đối tượng.
  - **Compile-Time Checking:** Cho phép kiểm tra tại thời điểm biên dịch nhằm giảm lỗi phát sinh trong quá trình chạy chương trình.

Example

Before Generics, we need to type cast.

```
ArrayList list = new ArrayList();
list.add("hello");
String s = (String) list.get(0);//typecasting
```

After Generics, we don't need to typecast the object.

```
ArrayList<String> list = new ArrayList<String>();
list.add("hello");
String s = list.get(0);
```

Methods

- ***.add***(Object x)
  Thêm mới 1 object x vào ArrayList
- ***.remove***(Object x)
  Loại bỏ 1 object x khỏi ArrayList
- ***.remove***(int index)
  Loại bỏ object x tại vị trí Index
- ***.get***(int index)
  Lấy phần tử x tại vị trí index
- ***.set***(int index, Object x)
  Cập nhật giá trị cho phần tử tại vị trí index bởi object x
- ***.contains***(Object o)
  Trả về kết quả True, nếu x có trong ArrayList
- ***.isEmpty***()
  Trả về True nếu ArrayList hiện đang trống rỗng
- ***.toArray***()
  Chuyển ArrayList thành 1 mảng Objets
- ***.clone***()
  Sao chép toàn bộ ArrayList hiện tại vào trong 1 ArrayList mới
- ***.clear***()
  Xóa toàn bộ nội dung của 1 ArrayList

Foreach in Java

- Được sử dụng để lặp trên 1 Array hay collection. Ví dụ: Duyệt qua các phần tử có trong ArrayList

```
for(data_type variable : array | collection)
{
    //-- do some things
    -----
}
```

- **Data_type**: kiểu dữ liệu của variable
- **Array | collection** : Tên của mảng, hay tên của Collection chứa dữ liệu cần duyệt

Foreach :: Array

```
class ForEachExample1{
    public static void main(String args[]){
        int arr[]={12,13,14,44};
        for(int i:arr){
            System.out.println(i);
        }
    }
}
```

Foreach :: ArrayList

```
import java.util.*;
class ForEachExample2{
    public static void main(String args[]){
        ArrayList<String> list=new ArrayList<String>();
        list.add("vimal");
        list.add("sonoo");
        list.add("ratan");
        for(String s:list){
            System.out.println(s);
        }
    }
}
```

Example

- Viết chương trình có các chức năng (cho người dùng chọn):

1. Nhập danh sách sinh viên
2. In danh sách sinh viên
3. Thêm sinh viên
4. Xóa sinh viên
5. Tìm sinh viên
0. Thoát

HD: Tạo class student, student management, dùng ArrayList để tạo danh sách SV, lưu trữ danh sách mà các object là student

Example

- Class SinhVien :: Thêm, xóa, truy xuất phần tử trên ArrayList
- Class SanPham :: Lặp trên danh sách sản phẩm
- Class PhuongTienGiaoThong :: Sinh viên tự làm
- Class NhanVien :: Sinh viên tự làm

Tham khảo

```java
import java.util.Scanner;

public class StudentManagement {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Student> studentList = new ArrayList<>();
        int choice;

        do {
            System.out.println("--------------");
            System.out.println("1. Nhập danh sách sinh viên");
            System.out.println("2. In danh sách sinh viên");
            System.out.println("3. Thêm sinh viên");
            System.out.println("4. Xóa sinh viên");
            System.out.println("5. Tìm sinh viên");
            System.out.println("0. Thoát");
            System.out.println("--------------");

            System.out.print("Lựa chọn của bạn: ");
            choice = scanner.nextInt();
            scanner.nextLine();
```

```java
switch (choice) {
    case 0:
        System.out.println("Tạm biệt!");
        break;

    case 1:
        // Nhập danh sách sinh viên
        System.out.print("Số lượng sinh viên cần nhập: ");
        int n = scanner.nextInt();
        scanner.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.println("Nhập thông tin sinh viên thứ " + (i+1) + ":");
            Student student = new Student();

            System.out.print("Mã số: ");
            student.setId(scanner.nextLine());

            System.out.print("Họ tên: ");
            student.setName(scanner.nextLine());

            System.out.print("Lớp: ");
            student.setClassName(scanner.nextLine());

            studentList.add(student);
        }

        break;
```

```java
case 2:
    // In danh sách sinh viên
    System.out.println("Danh sách sinh viên:");
    for (int i = 0; i < studentList.size(); i++) {
        System.out.println(studentList.get(i));
    }

    break;
```

```java
case 3:
    // Thêm sinh viên
    Student student = new Student();

    System.out.print("Mã số: ");
    student.setId(scanner.nextLine());

    System.out.print("Họ tên: ");
    student.setName(scanner.nextLine());

    System.out.print("Lớp: ");
    student.setClassName(scanner.nextLine());

    studentList.add(student);

    System.out.println("Thêm sinh viên thành công!");

    break;
```

```java
case 4:
    // Xóa sinh viên
    System.out.print("Nhập mã số sinh viên cần xóa: ");
    String id = scanner.nextLine();

    for (int i = 0; i < studentList.size(); i++) {
        if (studentList.get(i).getId().equals(id)) {
            studentList.remove(i);
            System.out.println("Xóa sinh viên thành công!");
            break;
        }
    }

    break;
```

```java
case 5:
    // Tìm sinh viên
    System.out.print("Nhập mã số sinh viên cần tìm: ");
    id = scanner.nextLine();

    for (int i = 0; i < studentList.size(); i++) {
        if (studentList.get(i).getId().equals(id)) {
            System.out.println("Thông tin sinh viên:");
            System.out.println(studentList.get(i));
            break;
        }
    }

    break;
```

```
                default:
                    System.out.println("Lựa chọn không hợp lệ!")
                    break;
            }
        } while (choice != 0);

        scanner.close();
    }
}
```

Array – ArrayList

	Array	ArrayList
Length:	Fixed Length	Variable Length
Data Types:	Primitives, Objects	Objects, Generics
Performance:	Get and Insert in O(1)	Get and Insert in O(1)

Similarities

- **add and get method :** phương thức add và get giống nhau.
- **Duplicate elements :** Có thể chứa các phần tử trùng nhau.
- **Null Values :** có thể chứa giá trị null.
- **Unordered :** Không sắp xếp.

Differences

- Khác biệt về truy xuất phần tử
- Để truy xuất đến các phần tử trên mảng, ta sử dụng cặp dấu []. Trong ArrayList, ta dùng phương thức .get()

```java
// A Java program to demonstrate differences between array
// and ArrayList
import java.util.ArrayList;
import java.util.Arrays;

class Test
{
    public static void main(String args[])
    {
        /* ........... Normal Array............. */
        int[] arr = new int[3];
        arr[0] = 1;
        arr[1] = 2;
        System.out.println(arr[0]);

        /*............ArrayList..............*/
        // Create an arrayList with initial capacity 2
        ArrayList<Integer> arrL = new ArrayList<Integer>(2);

        // Add elements to ArrayList
        arrL.add(1);
        arrL.add(2);

        // Access elements of ArrayList
        System.out.println(arrL.get(0));
    }
}
```

Differences

- Khác biệt về tính linh hoạt đối với kích thước
- Kích thước của mảng là cố định, không thể thay đổi kích thước sau khi khai báo
- Kích thước của ArrayList "co giãn" tùy theo nhu cầu sử dụng. Tự động tăng lên khi thêm mới, tự động giảm bớt khi bị loại bỏ

```java
// A Java program to demonstrate differences between array
// and ArrayList
import java.util.ArrayList;
import java.util.Arrays;
class Test
{
    public static void main(String args[])
    {
        /* ........... Normal Array............. */
        // Need to specify the size for array
        int[] arr = new int[3];
        arr[0] = 1;
        arr[1] = 2;
        arr[2] = 3;
        // We cannot add more elements to array arr[]

        /*............ArrayList..............*/
        // Need not to specify size
        ArrayList<Integer> arrL = new ArrayList<Integer>();
        arrL.add(1);
        arrL.add(2);
        arrL.add(3);
        arrL.add(4);
        // We can add more elements to arrL

        System.out.println(arrL);
        System.out.println(Arrays.toString(arr));
    }
}
```

Differences

- Khác biệt về đối tượng lưu trữ
- Mảng có thể chứa dữ liệu nguyên thủy như int, double, float, … hoặc cũng có thể chứa object
- ArrayList chỉ có thể chứa object, không chứa primitive data

```
import java.util.ArrayList;
class Test
{
    public static void main(String args[])
    {
        // allowed
        int[] array = new int[3];

        // allowed, however, need to be intialized
        Test[] array1 = new Test[3];

        // not allowed (Uncommenting below line causes
        // compiler error)
        // ArrayList<char> arrL = new ArrayList<char>();

        // Allowed
        ArrayList<Integer> arrL1 = new ArrayList<>();
        ArrayList<String> arrL2 = new ArrayList<>();
        ArrayList<object> arrL3 = new ArrayList<>();
    }
}
```

Câu hỏi

Array List trong Java là gì?

A) Một danh sách liên kết

B) Một mảng động

C) Một danh sách liên kết kép

D) Một mảng tĩnh

Câu hỏi

Array List trong Java là gì?

A) Một danh sách liên kết

B) Một mảng động

C) Một danh sách liên kết kép

D) Một mảng tĩnh

Câu hỏi

Để khởi tạo một Array List trong Java, ta sử dụng từ khóa nào?

A) ArrayList

B) Array_List

C) Array-List

D) arraylist

Câu hỏi

Để khởi tạo một Array List trong Java, ta sử dụng từ khóa nào?

A) ArrayList

B) Array_List

C) Array-List

D) arraylist

Câu hỏi

Khi muốn thêm một phần tử vào cuối danh sách trong Array List, ta sử dụng phương thức nào?

A) addLast()

B) addFirst()

C) add()

D) addEnd()

Câu hỏi

Khi muốn thêm một phần tử vào cuối danh sách trong Array List, ta sử dụng phương thức nào?

A) addLast()

B) addFirst()

C) add()

D) addEnd()

Câu hỏi

Phương thức nào trong Array List được sử dụng để lấy phần tử tại vị trí xác định trong danh sách?

A) get()

B) fetch()

C) retrieve()

D) obtain()

Câu hỏi

Phương thức nào trong Array List được sử dụng để lấy phần tử tại vị trí xác định trong danh sách?

A) get()

B) fetch()

C) retrieve()

D) obtain()

Câu hỏi

Để lấy kích thước của một Array List, ta sử dụng phương thức nào?

A) getSize()

B) length()

C) size()

D) count()

Câu hỏi

Để lấy kích thước của một Array List, ta sử dụng phương thức nào?

A) getSize()

B) length()

C) size()

D) count()

Nhớ gì ?!!!

- Array, ArrayList dùng để làm gì ? Truy xuất phần tử ra sao ?
- Array và ArrayList giống & khác nhau thế nào ?
- Các phương thức tiêu biểu của ArrayList
- Cú pháp & cách sử dụng foreach trong duyệt dữ liệu Array, hay ArrayList

Bài tập

- Bài tập 1: Viết chương trình Java để thực hiện các thao tác cơ bản với một Array List. Chương trình sẽ cho phép người dùng thêm, xóa và in ra các phần tử trong danh sách.

Array - ArrayList

Môn học: Kỹ thuật lập trình

GV: TS. Võ Thị Hồng Thắm